

The Game of Life: Simulating Cell's Lives

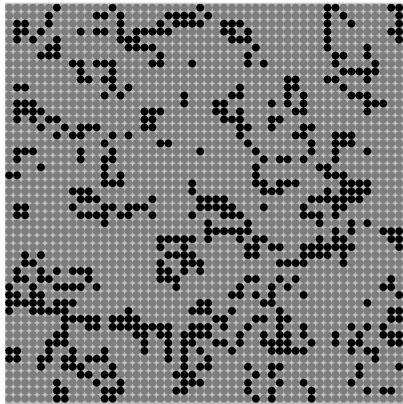
Abstract:

In this project, I created a game of life simulation, where there is a grid of cells and the life of a cell depends on how many of their neighbors are alive or dead. Only if two or three of the cell's neighbors are alive, the cell stays/becomes alive. I also created a display to visualize this, and show what happens when this algorithm is run many times. A key concept I used for this is creating grids using 2D arrays. This allowed me to store cells in a grid and keep track of their neighbors. I then ran experiments to see how different variables such as, grid size and the probability that a cell is initially alive or dead, affected the number of cells left after the simulation was run many times.

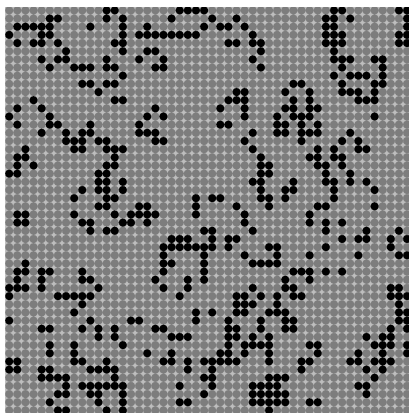
Results:

Here is an example of a timestep where we have the grid and then the grid after the algorithm that determines whether or not a cell stays alive is run once.

Before the algorithm is run:



After the algorithm is run:



You can see how the cells which are black (which means they are alive), change and some become gray.

Experiments from Exploration:

I ran two experiments for this project. One in which I varied the initial status chance. The initial status chance is the probability that a cell is initially alive or dead. So it determines the state of each cell on the landscape or grid. For example if the initial status chance was 80% then each cell has a 80% chance of being alive on the initial grid.

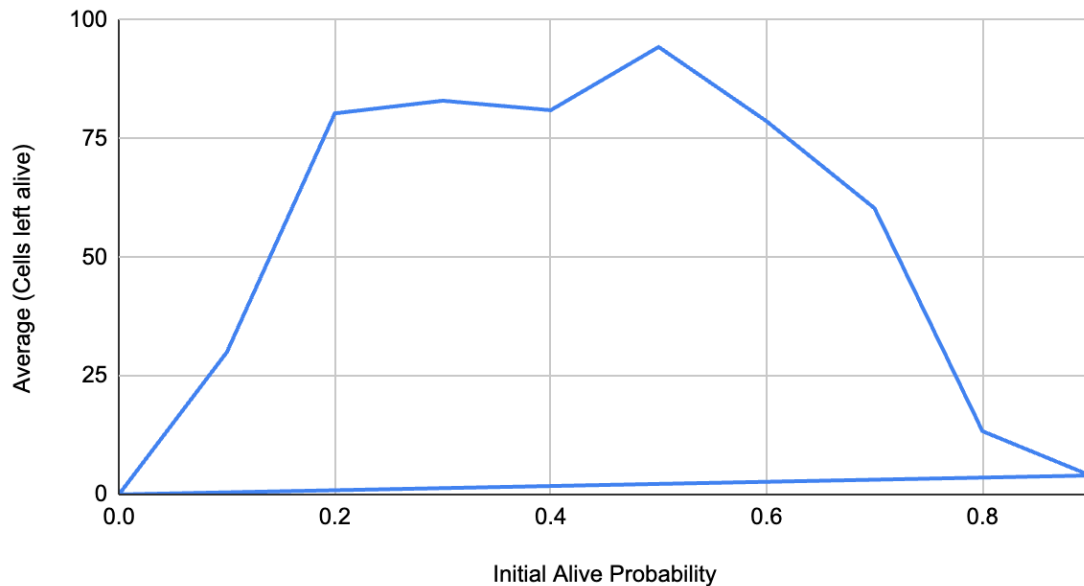
Experiment 1: In experiment One I varied the initial status chance from 0 to 1 in 0.1 increments. While I had the number of timesteps (number of times the algorithm determining whether each cell stays alive) kept at 1000 and the grid size kept at 50 x 50. Below was my hypothesis.

Hypothesis: If the initial status chance is increased and the grid dimensions stay the same then the number of living cells after a sufficiently long time will be greater.

Initial Alive Probability	Average of the three Rounds (Cells left alive)	Trial One (Cells left Alive)	Trial Two (Cells left Alive)	Trial Three (Cells left Alive)
0	0	0	0	0
0.1	30	30	71	70
0.2	80.333	36	97	108
0.3	83	105	68	73
0.4	81	79	68	96
0.5	94.333	89	86	108
0.6	78.667	84	98	54
0.7	60.333	37	67	77
0.8	13.333	8	8	24
0.9	4	0	4	8
1	0	0	0	0

This graph shows the results of the experiment run. I did three trials and averaged them out. My hypothesis was wrong. Here you can clearly see that when the initial alive probability is at 0.5, is when on average the number of cells left was the greatest, but as it increases towards one, the average number of cells left after 1000 time steps decreases.

Average (Cells left alive) vs. Initial Alive Probability



This graph reflects the results shown in the table. Here we also see that the greatest number of cells left is when the initial alive probability is at 0.5 or closer to the middle between 0 and 1. This proves my hypothesis wrong because the average number of cells left does not increase as the initial alive probability increases, because the average number of cells left, increases and then decreases.

Experiment 2:

In experiment two, I changed the grid sizes to see how grid size affects the number of cells left. The grid ranges from size 10 x 10 to 50 x 50 with initial status chance at 0.25 with 1000 iterations. Below is my hypothesis.

Hypothesis:

As the grid increases in size the average number of cells left will also increase.

	Number of Cells Left			
Grid Size	Averages	Trial 1	Trial 2	Trial 3
10x10	5.333	0	12	4
20x20	8.333	11	11	7
30x30	34	40	28	34
40x40	55	68	55	42
50x50	81	68	95	80

This table shows my results from the experiments I ran. I first ran the simulation with the grid size at 10x10, the initial status chance at 0.25, and did 1000 iterations. Then for each new grid size I kept the other variables constant. This data shows how my hypothesis is supported, and as the grid size increases, the average number of cells left also clearly increases.

Extension:

For my extension, I added to the LifeSimulation.java file, and added code that allows the user to put in command line arguments to adjust the grid size, initial status chance and the number of iterations. I also created a README file that explains how to use the command line parameters correctly, and put in command line arguments and get output that reflects what the user put in. Here is an example of what the user would type on the command line and what would be the output.

```
java LifeSimulation 50 50 0.25 10
```

This would run the simulation 10 times, in a grid size 50 x 50, and with an initial status chance of 0.25.

Acknowledgements:

My only Acknowledgments are imported libraries which are:

java.util.ArrayList

java.awt.Color

java.art.Graphics